



flux One

PLAN DE TRABAJO

Instalación de Apache Kafka y Dashboard de Detección Oportuna
Plataforma FLUX – Infraestructura EC2



1. Resumen Ejecutivo

1. La instalación de Apache Kafka en modalidad dockerizada con modo KRaft es la solución técnicamente óptima para el entorno EC2 de FLUX. El análisis real del consumo de recursos confirma que el servidor tiene capacidad suficiente para absorber el nuevo servicio sin comprometer la estabilidad de los microservicios existentes. La integración con Grafana permitirá construir un sistema de observabilidad completo, con detección oportuna de fallos en transacciones, tiempos de respuesta, logs críticos y tráfico HTTP, tanto a nivel general como por microservicio individual.

Tiempo total estimado de implementación: 5 a 6 horas en una jornada de trabajo.



2. Análisis de la Infraestructura Existente

2.1 Especificaciones del servidor EC2

Procesador	Intel® Xeon® Platinum 8259CL @ 2.50GHz — 2 vCPU (1 core, 2 threads, Hyper-Threading)
Memoria RAM	3.744 GiB total — arquitectura 64-bit, soporte AVX-512
Almacenamiento	48 GB en /dev/root — 18 GB disponibles (64% de uso actual)
Virtualización	KVM (Hypervisor) — instancia AWS EC2
Sistema base	Ubuntu 24 LTS — Docker Engine instalado y operativo

2.2 Estado actual de recursos

Tras analizar el consumo real de los 18 contenedores Docker en ejecución, se obtuvo el siguiente balance de recursos:

RECURSO	CAPACIDAD	CONSUMO	DISPONIBLE	EVALUACIÓN
Memoria RAM	3.7 GB	~900 MB	~2.8 GB	Funcionamiento estable
Procesador (vCPU)	2 vCPU	Moderado	Disponible	Rendimiento adecuado
Almacenamiento (/)	48 GB	31 GB	18 GB	Espacio suficiente



* El uso real de RAM de los contenedores Docker suma aproximadamente 400 MB. El valor de 2.3 GB reportado por `free -h` incluye buffers y caché del sistema operativo, memoria que se libera cuando es necesario.

2.3 Contenedores actualmente en ejecución

El ecosistema FLUX cuenta con 18 servicios dockerizados operativos:

- Infraestructura base: PostgreSQL 16, Redis 7, RabbitMQ 3.13, Nginx 1.27
- Core del sistema: flux_scheduler, flux_worker, flux_app
- Microservicios de negocio: adapters, biller, accounts, bill-inquiry, bill-payment, reversal, reconciliation, receipt, fee, audit, fraud-risk

3. Justificación de la Solución Seleccionada

3.1 Opciones evaluadas

Se evaluaron tres alternativas para la implementación de Kafka:

Opción	Ventajas	Desventajas / Riesgos
Kafka dockerizado (KRaft)	Integración nativa con docker-compose existente. Sin Zookeeper (modo KRaft). Fácil gestión y despliegue.	Requiere control del límite de memoria (mitigado con mem_limit: 512m).
Kafka como servicio nativo	Aislamiento total del motor de Kafka del ecosistema Docker.	JVM pesada (~400 MB solo de base). Dificulta la portabilidad y el ciclo de vida.
Kafka en equipo local + tunnel	No consume recursos del EC2.	Depende de la disponibilidad del equipo local. Latencia alta e inestable. No apto para producción.



3.2 Conclusión de la evaluación

La opción seleccionada es Kafka dockerizado en modo KRaft por las siguientes razones técnicas concretas:

- El consumo real de RAM de los contenedores existentes es de aproximadamente 400 MB, dejando aproximadamente 2.8 GB disponibles para nuevos servicios.
- Kafka en modo KRaft elimina la dependencia de Zookeeper, reduciendo el footprint (cantidad de recursos consumidos por aplicación) de memoria a aproximadamente 300–400 MB con el límite de 512 MB configurado.
- El disco dispone de 18 GB libres, más que suficiente para logs y datos de Kafka en entorno de desarrollo/staging.
- La integración con el docker-compose existente simplifica el ciclo de vida, el monitoreo y la consistencia del entorno.
- La imagen bitnami/kafka es la más liviana y estable para entornos con recursos limitados.

4. Plan de Instalación — Kafka (KRaft Mode)

4.1 Cronograma de fases

#	Fase	Actividades	Tiempo Est.
1	Preparación del entorno	Verificar puertos libres, configurar variables de entorno, crear directorio de datos para Kafka	30 min
2	Despliegue de Kafka (KRaft)	Agregar servicio Kafka al docker-compose, configurar límite de memoria (512MB), levantar contenedor	20 min
3	Validación de Kafka	Verificar logs del contenedor, crear topic de prueba, producir y consumir mensaje de validación	20 min
4	Instalación de Grafana	Agregar Grafana al docker-compose con volumen persistente, exponer puerto 3009	15 min
5	Integración Kafka-Grafana	Instalar plugin kafka-datasource en Grafana, configurar datasource apuntando a Kafka	30 min
6	Instrumentación de MS	Configurar microservicios para producir eventos a topics de Kafka, validar flujo de datos	60 min
7	Creación de Dashboards	Construir dashboards de detección oportuna según especificación, por microservicio	90 min
8	Pruebas y ajustes	Validar datos en tiempo real, ajustar queries, configurar alertas básicas	45 min



4.2 Configuración técnica de Kafka

El siguiente bloque debe agregarse al docker-compose.yml del entorno FLUX:

```
kafka:
  image: bitnami/kafka:latest
  container_name: flux_kafka
  ports:
    - "9092:9092"
  environment:
    - KAFKA_CFG_NODE_ID=1
    - KAFKA_CFG_PROCESS_ROLES=broker,controller
    - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
    - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092
    - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1 @kafka:9093
    - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
  mem_limit: 512m # Protege la RAM del EC2
  restart: unless-stopped
  volumes:
    - kafka_data:/bitnami/kafka
```

Nota: Agregar también el volumen kafka_data en la sección volumes: del docker-compose.yml para persistencia de datos

4.3 Comandos de validación post-instalación

1. Verificar que el contenedor esté healthy: `docker ps | grep flux_kafka`
2. Revisar logs de arranque: `docker logs flux_kafka --tail 50`
3. Crear topic de prueba: `docker exec flux_kafka kafka-topics.sh --create --topic test --bootstrap-server localhost:9092 --partitions 1 --replication-factor 1`
4. Producir mensaje de prueba y verificar consumo para confirmar que el broker funciona correctamente.



5. Dashboard de Detección Oportuna — Grafana

5.1 Objetivo del dashboard

El dashboard de detección oportuna tiene como propósito centralizar en tiempo real los indicadores críticos de la plataforma FLUX, permitiendo identificar de forma temprana fallos, degradaciones de rendimiento y anomalías en el tráfico antes de que impacten al usuario final.

5.2 Paneles del dashboard principal

Panel	Métricas / Visualización	Fuente de datos
Transacciones Aprobadas	Conteo en tiempo real, tasa por minuto, % aprobación	Topic: transactions.approved
Transacciones Fallidas	Conteo de fallos, tipos de error, tendencia temporal	Topic: transactions.failed
Tiempos de Respuesta MS	P50 / P95 / P99 latencia por microservicio, heatmap	Topic: ms.response-times
Logs Críticos MS	Errores, excepciones, warnings por servicio, stream en vivo	Topic: ms.logs
Peticiones Nginx	Total req/min, distribución códigos HTTP (200/4xx/5xx)	Topic: nginx.access-logs
Dashboard por MS (x12)	Métricas individuales: latencia, errores, throughput	Topic: ms.



5.3 Detalle por panel

Panel 1 — Transacciones Aprobadas y Fallidas

- Gráfico de barras apiladas: aprobadas vs. fallidas por intervalo de tiempo (1 min / 5 min / 1 hora).
- Indicador de tasa de aprobación en tiempo real (gauge %).
- Alerta cuando tasa de fallo supere umbral configurable (por defecto > 5%).
- Fuente: topics Kafka producidos por bill-payment-service y adapters-service.

Panel 2 — Tiempos de Respuesta de Microservicios

- Heatmap de latencia por microservicio (P50, P95, P99).
- Línea de tiempo con degradaciones históricas de los últimos 30 minutos.
- Identificación visual de outliers (microservicios fuera del SLA).
- Fuente: cada microservicio publica su tiempo de respuesta al topic ms.response-times.

Panel 3 — Logs Críticos de Microservicios

- Stream en tiempo real de logs con nivel ERROR y WARN.
- Filtro por microservicio, tipo de error y rango de tiempo.
- Conteo de errores por servicio en la última hora (ranking).
- Fuente: topic ms.logs — los MS publican sus logs estructurados en JSON.

Panel 4 — Peticiones Nginx y Códigos de Respuesta

- Total de peticiones por minuto (req/min).
- Distribución de códigos HTTP: 200, 201, 400, 401, 403, 404, 500, 502, 503.
- Alerta cuando respuestas 5xx superen el 1% del total de peticiones.
- Fuente: Nginx configurado para publicar access logs al topic nginx.access-logs.



Panel 5 — Dashboards Individuales por Microservicio

Se creará un dashboard dedicado para cada uno de los 12 microservicios:

- adapters-service (puerto 3000)
- biller-service (puerto 3001)
- accounts-service / wallet-ledger (puerto 3002)
- bill-inquiry-service (puerto 3003)
- bill-payment-service (puerto 3004)
- reconciliation-service (puerto 3005)
- reversal-service (puerto 3006)
- receipt-service (puerto 3007)
- fee-service (puerto 3008)
- fraud-risk-service (puerto 3010)
- audit-service (puerto 3011)
- flux_app (puerto 4444)

Cada dashboard individual incluirá: throughput de peticiones, latencia promedio, tasa de errores, estado de salud del contenedor y últimos logs críticos.

6. Consideraciones de Seguridad y Operación

- Kafka correrá en modo PLAINTEXT internamente (sin TLS) dado que es una red Docker privada. Si se expone externamente, se debe configurar SASL/SSL.
- El límite de memoria de 512 MB en Kafka es un control de seguridad para evitar que un pico de carga afecte los demás microservicios.
- Se recomienda monitorear el uso de swap durante las primeras 48 horas post-instalación con: `watch -n 5 free -h`
- Los topics de Kafka deben configurarse con retención limitada (ej. 24h) para controlar el crecimiento del disco.
- Grafana debe desplegarse con volumen persistente para preservar dashboards ante reinicios.